#Extraction of data from GDC portal

```
import requests
import re

file_id = "f6529292-5452-49e5-9e69-61da6048fb7e"

data_endpt = "https://api.gdc.cancer.gov/data/{}".format(file_id)

response = requests.get(data_endpt, headers = {"Content-Type":
"application/json"})

# The file name can be found in the header within the Content-Disposition
key.
response_head_cd = response.headers["Content-Disposition"]

file_name = re.findall("filename=(.+)", response_head_cd)[0]

with open(file_name, "wb") as output_file:
   output_file.write(response.content)
```

#Defining

```
import os
import subprocess
import requests
import jsona
import sys
import pandas as pd

import ssl
if (not os.environ.get('PYTHONHTTPSVERIFY', '') and getattr(ssl,
'_create_unverified_context', None)):
   ssl._create_default_https_context = ssl._create_unverified_context




#from tcga downloader import *
```

```
#ids=process_manifests(manifest_file)

'''
Match tumor and normal samples to a patient through GDC API. This will
provide metadata that
for associated files including sample type(tumor /normal) patient ID(TCGA
barcode), etc

The general query structure will look like this:

curl
"https://gdc-api.nci.nih.gov/files?size=100000&format=tsv&filters=XXXX&fie
lds=YYYY"

filters= :

The XXXX will be replaced with a URL-encoded JSON query that will filter a
list of files. The JSON query can be generated yourself and then URL
encoded, or the filtering can be performed on the GDC Data Portal
(https://gdc-portal.nci.nih.gov/search/s?facetTab=files). You can copy and
paste the string that appears where the * are displayed in the following:

https://gdc-portal.nci.nih.gov/search/f?filters=**********&facetTab=cases

'''

def prepare_payload(ids,data_type=None):

   #"Gene Expression Quantification"

   "Workflow Type" "HTSeq-Counts"

   "Data Category" "transcriptome profilling"
   "Experimental Strategy" "RNA-Seq"

   no_of_samples=len(ids)
   part1='''{
   "filters":{
       "op":"and",
       "content":[
```

```
            {
                "op":"in",
                "content":{
                    "field":"files.file_id",
                    "value":[%s]
                }
            },
            {
                "op":"=",
                "content":{
                    "field":"files.data_type",
                    "value":"%s"
                }
            }
        ]
    },'''%(",\n".join(ids),data_type)


    part2='''
"format":"TSV","fields":"file_id,file_name,cases.submitter_id,cases.diseas
e_type,cases.case_id,data_category,data_type,cases.samples.tumor_descripto
r,cases.samples.tissue_type,cases.samples.sample_type,cases.samples.submit
ter_id,cases.samples.sample_id,cases.samples.portions.analytes.aliquots.al
iquot_id,cases.samples.portions.analytes.aliquots.submitter_id","size":"%d
"}'''%no_of_samples

    payload_command='%s %s'%(part1,part2)

    payloadfile='payloadv3.txt'
    with open(payloadfile,'w') as output_:
        output_.write(payload_command)
    return payloadfile


def get_ids(manifest):
    try:
#manifest='all.txt'
        with open (manifest,'r') as input_:
            ids=["\"%s\""%i.strip('\n').split('\t')[0] for i in input_][1:]
```

```
        no_of_samples=len(ids)
        return ids
    except Exception as ex:
        return None
    #print(ids)


def get_metadata(payloadfile):
    metadata='Metadata.tsv'
    webaddr='\'https://api.gdc.cancer.gov/files\''
    args=['curl', '--request POST', '--header','\"Content-Type:
application/json\"',
      '--data','@%s'%payloadfile,webaddr, '>', metadata]

    single=' '.join(args)
    print(single)
    os.system(single)
    return metadata


#def get_metadatada():
#   ids=process_manifest()
 #  if ids==None:
  #     print('Error encountered\nPlease ensure that you are using the
correct manifest file')
  # else:
  #     payloadfile=prepare_payload(ids)
  #     download_data(payloadfile)

def download_data(metadatafile,sep='\t',outdir='downloads'):
    df=pd.DataFrame()
    data_df=pd.read_csv(metadatafile,sep=sep)
    sampletypes=data_df['cases.0.samples.0.sample_type'].unique()
    sampletypes=sampletypes.tolist()
    curr_dir=os.getcwd()

    if not os.path.exists(outdir):
        os.mkdir(outdir)
    else:
        print('output directory exists\ndata may be overwritten')
```

```
   for sampletype in sampletypes:

sel=data_df[data_df['cases.0.samples.0.sample_type'].str.contains(samplety
pe)][['file_id',

'file_name']]

       sampledir="%s/%s"%(outdir,sampletype)

       if not os.path.exists(sampledir):
           os.mkdir(sampledir)
       else:
           print('sample type directory exists\ndata may be overwritten')


       os.chdir(sampledir)
       all_file_ids=sel['file_id'].values
       download_list=[]
       for file_id in all_file_ids:
           args=['curl', '--remote-name', '--remote-header-name',
                 '\'https://api.gdc.cancer.gov/data/%s\''%file_id]
           cmd=' '.join(args)
           print('downloading %s'%file_id)
           os.system(cmd)
           #download_list.append(' '.join(args))
       os.chdir(curr_dir)
   print('Download complete\nAll data has been downloaded to
------------->%s'%outdir)
```

#Downloading metadata

```
ids=get_ids('TET_exp.txt')
len(ids)
payload=prepare_payload(ids,data_type='Gene Expression Quantification')
metadata=get_metadata(payload)
download_data(metadata,sep='\t',outdir='TARGET_AML')
payload=prepare_payload(ids,data_type='Gene Expression Quantification')
```

```
metadata=get_metadata(payload)
download_data(metadata,sep='\t',outdir='TARGET_AML')
```

```
#creating genematrix

import pandas as pd
import os
import glob

# Path to your data files (update the path and extension as needed)
file_paths = glob.glob("*.csv")  # adjust if using .tsv or another format

# Step 1: Extract column 2 and 3 from file1
file1 = 'file1.csv'
df_file1 = pd.read_csv(file1)

# Columns 2 and 3 are index 1 and 2
df_base = df_file1.iloc[:, [1, 2]].copy()

# Step 2: Extract column 7 (index 6) from each file and rename
df_combined = pd.DataFrame()

for path in file_paths:
    df = pd.read_csv(path)
    col7 = df.iloc[:, 6]  # column 7 is index 6
    basename = os.path.basename(path)
    df_combined[basename] = col7

# Step 3: Combine df_base and df_combined
final_df = pd.concat([df_base, df_combined], axis=1)

# Optional: Save to new CSV
# final_df.to_csv('combined_output.csv', index=False)

print(final_df)
```

#CIBERSORT USING R

```
# CIBERSORT R script v1.02 (last updated 02-02-2015)
# Note: Signature matrix construction is not currently available; use java version for full
functionality.
# Author: Aaron M. Newman, Stanford University (amnewman@stanford.edu)
# Requirements:
#       R v3.0 or later. (dependencies below might not work properly with earlier versions)
#       install.packages('e1071')
#       install.pacakges('parallel')
#       install.packages('preprocessCore')
#       if preprocessCore is not available in the repositories you have selected, run the following:
#           source("http://bioconductor.org/biocLite.R")
#           biocLite("preprocessCore")
# Windows users using the R GUI may need to Run as Administrator to install or update
packages.
# This script uses 3 parallel processes.  Since Windows does not support forking, this script will
run
# single-threaded in Windows.
#
# Usage:
#       Navigate to directory containing R script
#
#   In R:
#       source('CIBERSORT.R')
#       results <- CIBERSORT('sig_matrix_file.txt','mixture_file.txt', perm, QN)
#
#       Options:
```

```
#       i)  perm = No. permutations; set to >=100 to calculate p-values (default = 0)
#       ii) QN = Quantile normalization of input mixture (default = TRUE)
#
# Input: signature matrix and mixture file, formatted as specified at
http://cibersort.stanford.edu/tutorial.php
# Output: matrix object containing all results and tabular data written to disk
'CIBERSORT-Results.txt'
# License: http://cibersort.stanford.edu/CIBERSORT_License.txt


#dependencies
library(e1071)
library(parallel)
library(preprocessCore)

#Core algorithm
CoreAlg <- function(X, y){

    #try different values of nu
    svn_itor <- 3

    res <- function(i){
        if(i==1){nus <- 0.25}
        if(i==2){nus <- 0.5}
        if(i==3){nus <- 0.75}
        model<-svm(X,y,type="nu-regression",kernel="linear",nu=nus,scale=F)
        model
    }

    if(Sys.info()['sysname'] == 'Windows') out <- mclapply(1:svn_itor, res, mc.cores=1) else
    out <- mclapply(1:svn_itor, res, mc.cores=svn_itor)

    nusvm <- rep(0,svn_itor)
    corrv <- rep(0,svn_itor)

    #do cibersort
    t <- 1
    while(t <= svn_itor) {
        weights = t(out[[t]]$coefs) %*% out[[t]]$SV
        weights[which(weights<0)]<-0
        w<-weights/sum(weights)
        u <- sweep(X,MARGIN=2,w,'*')
        k <- apply(u, 1, sum)
        nusvm[t] <- sqrt((mean((k - y)^2)))
```

```
        corrv[t] <- cor(k, y)
        t <- t + 1
    }

    #pick best model
    rmses <- nusvm
    mn <- which.min(rmses)
    model <- out[[mn]]

    #get and normalize coefficients
    q <- t(model$coefs) %*% model$SV
    q[which(q<0)]<-0
    w <- (q/sum(q))

    mix_rmse <- rmses[mn]
    mix_r <- corrv[mn]

    newList <- list("w" = w, "mix_rmse" = mix_rmse, "mix_r" = mix_r)

}

#do permutations
doPerm <- function(perm, X, Y){
    itor <- 1
    Ylist <- as.list(data.matrix(Y))
    dist <- matrix()

    while(itor <= perm){
        #print(itor)

        #random mixture
        yr <- as.numeric(Ylist[sample(length(Ylist),dim(X)[1])])

        #standardize mixture
        yr <- (yr - mean(yr)) / sd(yr)

        #run CIBERSORT core algorithm
        result <- CoreAlg(X, yr)

        mix_r <- result$mix_r

        #store correlation
        if(itor == 1) {dist <- mix_r}
        else {dist <- rbind(dist, mix_r)}
```

```
        itor <- itor + 1
    }
    newList <- list("dist" = dist)
}

#main function
CIBERSORT <- function(sig_matrix, mixture_file, perm=0, QN=TRUE){

    #read in data
    X <- read.table(sig_matrix,header=T,sep="\t",row.names=1,check.names=F)
    Y <- read.table(mixture_file, header=T, sep="\t", row.names=1,check.names=F)

    X <- data.matrix(X)
    Y <- data.matrix(Y)

    P <- perm #number of permutations

    #anti-log if max < 50 in mixture file
    if(max(Y) < 50) {Y <- 2^Y}

    #quantile normalization of mixture file
    if(QN == TRUE){
        tmpc <- colnames(Y)
        tmpr <- rownames(Y)
        Y <- normalize.quantiles(Y)
        colnames(Y) <- tmpc
        rownames(Y) <- tmpr
    }

    #intersect genes
    Xgns <- row.names(X)
    Ygns <- row.names(Y)
    YintX <- Ygns %in% Xgns
    Y <- Y[YintX,]
    XintY <- Xgns %in% row.names(Y)
    X <- X[XintY,]

    #standardize sig matrix
    X <- (X - mean(X)) / sd(as.vector(X))

    #empirical null distribution of correlation coefficients
    if(P > 0) {nulldist <- sort(doPerm(P, X, Y)$dist)}
```

```
    #print(nulldist)

    header <- c('Mixture',colnames(X),"P-value","Correlation","RMSE")
    #print(header)

    output <- matrix()
    itor <- 1
    mixtures <- dim(Y)[2]
    pval <- 9999

    #iterate through mixtures
    while(itor <= mixtures){

        y <- Y[,itor]

        #standardize mixture
        y <- (y - mean(y)) / sd(y)

        #run SVR core algorithm
        result <- CoreAlg(X, y)

        #get results
        w <- result$w
        mix_r <- result$mix_r
        mix_rmse <- result$mix_rmse

        #calculate p-value
        if(P > 0) {pval <- 1 - (which.min(abs(nulldist - mix_r)) / length(nulldist))}

        #print output
        out <- c(colnames(Y)[itor],w,pval,mix_r,mix_rmse)
        if(itor == 1) {output <- out}
        else {output <- rbind(output, out)}

        itor <- itor + 1

    }

    #save results
    write.table(rbind(header,output), file="CIBERSORT-Results.txt", sep="\t", row.names=F,
col.names=F, quote=F)

    #return matrix object containing all results
    obj <- rbind(header,output)
```

```
  obj <- obj[,-1]
  obj <- obj[-1,]
  obj <- matrix(as.numeric(unlist(obj)),nrow=nrow(obj))
  rownames(obj) <- colnames(Y)
  colnames(obj) <- c(colnames(X),"P-value","Correlation","RMSE")
  obj
}
setwd("~/Documents/cibersort_directory")
source("CIBERSORT.R")
result <- CIBERSORT("LM22.txt", "TET2_curated.tsv", perm=1000, QN= FALSE)
head(result)
# Save the result as a TSV file
write.table(result, "TET2_immune_profile.tsv", sep = "\t", row.names = TRUE, col.names = NA,
quote = FALSE)
```

#LIMMA ANALYSIS and VOLCANO PLOT

```
import pandas as pd
import numpy as np
from statsmodels.formula.api import ols
from statsmodels.stats.multitest import multipletests
from tqdm import tqdm

### STEP 1: Load full expression matrix ###
df = pd.read_csv("/Users/kushaldas/Downloads/final_output2.csv", sep="\t")

# Extract TET2 row and drop 'gene_name' column
tet2_expr = df[df['gene_name'] == 'TET2'].iloc[0, 1:]

# Calculate TET2 quantiles
q25, q75 = tet2_expr.quantile([0.25, 0.75])

# Get high and low expressing sample names
hs_samples = tet2_expr[tet2_expr > q75].index.tolist()
ls_samples = tet2_expr[tet2_expr < q25].index.tolist()
selected_samples = hs_samples + ls_samples

### STEP 2: Filter counts matrix to just HS and LS samples ###
# Subset df to selected columns + gene_name
filtered_df = df[['gene_name'] + selected_samples]
```

```
# Save filtered matrix if needed
filtered_df.to_csv("/Users/kushaldas/Downloads/TET2_selected_samples_only.csv",
index=False)

# Prepare counts matrix for modeling
counts = filtered_df.set_index('gene_name').T  # shape = (70 samples, ~genes)

### STEP 3: Create Metadata ###
metadata = pd.DataFrame({
    'Sample': selected_samples,
    'group': ['HS'] * len(hs_samples) + ['LS'] * len(ls_samples)
}).set_index('Sample')

# Align metadata and counts
metadata['group'] = pd.Categorical(metadata['group'], categories=['LS', 'HS'])
counts = counts.loc[metadata.index]
counts['group'] = metadata['group']

### STEP 4: Linear modeling + DE analysis ###
# Log2-transform numeric values (except 'group')
counts_numeric = np.log2(counts.drop(columns='group') + 1)
counts_numeric['group'] = counts['group']

results = []
gene_columns = counts_numeric.columns.drop('group')

for gene in tqdm(gene_columns, desc="Analyzing genes"):
    try:
        model_df = pd.DataFrame({
            'expr': counts_numeric[gene],
            'group': counts_numeric['group']
        })

        if model_df['group'].nunique() < 2 or model_df['expr'].var() == 0:
            continue

        model = ols("expr ~ group", data=model_df).fit()
        contrast_term = [k for k in model.params.keys() if k.startswith('group[T.')]
        if contrast_term:
            coef = model.params[contrast_term[0]]
            stat = model.tvalues[contrast_term[0]]
            pval = model.pvalues[contrast_term[0]]
        else:
            coef, stat, pval = np.nan, np.nan, np.nan
```

```
        base_mean = model_df['expr'].mean()
        results.append([gene, base_mean, coef, stat, pval])

    except Exception as e:
        print(f"⚠️ {gene} failed: {e}")
        results.append([gene, np.nan, np.nan, np.nan, np.nan])

# Create DE results DataFrame
res_df = pd.DataFrame(results, columns=['Geneid', 'baseMean', 'log2FoldChange', 'stat',
'pvalue'])

# Add padj using FDR correction (only for valid p-values)
valid_pvals = res_df['pvalue'].notnull()
padj = np.full(len(res_df), np.nan)
padj[valid_pvals] = multipletests(res_df.loc[valid_pvals, 'pvalue'], method='fdr_bh')[1]
res_df['padj'] = padj

# Sort and export
res_df = res_df.sort_values('padj')
high_fc_genes = res_df[(res_df['pvalue'] < 0.01) & (abs(res_df['log2FoldChange']) > 1)]
high_fc_genes.to_csv('/Users/kushaldas/Downloads/TET2_high_FC_only.csv',
index=False)#used for GSEA
res_df.to_csv('/Users/kushaldas/Downloads/limma_style_TET2_results.csv', index=False)

filtered_res = res_df[
    (res_df['pvalue'] < 0.01) &
    (res_df['log2FoldChange'] > -1) &
    (res_df['log2FoldChange'] < 1)
]

# Show the filtered result
print(filtered_res)

# Optionally save it
filtered_res.to_csv('/Users/kushaldas/Downloads/TET2_filtered_results.csv', index=False)
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Replace inf or -inf values if any
res_df = res_df.replace([np.inf, -np.inf], np.nan).dropna(subset=['log2FoldChange', 'pvalue'])

# Create -log10(pvalue) column
```

```
res_df['neg_log10_pvalue'] = -np.log10(res_df['pvalue'])

# Define significance condition
res_df['significant'] = (
    (res_df['pvalue'] < 0.01) &
    (abs(res_df['log2FoldChange']) > 1)
)

# Plot
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=res_df,
    x='log2FoldChange',
    y='neg_log10_pvalue',
    hue='significant',
    palette={True: 'red', False: 'gray'},
    edgecolor=None,
    alpha=0.8
)

# Add horizontal and vertical threshold lines
plt.axhline(-np.log10(0.01), color='green', linestyle='--', linewidth=1)
plt.axvline(1, color='blue', linestyle='--', linewidth=1)
plt.axvline(-1, color='blue', linestyle='--', linewidth=1)

# Annotate TET2 if present
if 'TET2' in res_df['Geneid'].values:
    tet2_row = res_df[res_df['Geneid'] == 'TET2'].iloc[0]
    plt.scatter(tet2_row['log2FoldChange'], tet2_row['neg_log10_pvalue'],
                color='black', s=100, label='TET2', zorder=5)
    plt.text(tet2_row['log2FoldChange'], tet2_row['neg_log10_pvalue'] + 0.2,
             'TET2', ha='center', fontsize=9, color='black')

# Labels and title
plt.title("Volcano Plot - TET2 High vs Low Expression")
plt.xlabel("log2(Fold Change)")
plt.ylabel("-log10(p-value)")
plt.legend(title='Gene Status', loc='upper right')
plt.tight_layout()
plt.show()
```

#bar plot(mean Vs standard deviation)[removed last three columns]

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import ttest_ind
df = pd.read_csv("TET2_immune_profile2.csv")
df = df.loc[:, ~df.columns.str.contains('Unnamed', case=False)]
if df.columns[0].lower() in ['sampleid', 'sample', 'id', 'samples']:
    df.set_index(df.columns[0], inplace=True)
df = df.apply(pd.to_numeric, errors='coerce')
df.dropna(how='all', inplace=True)
if df.shape[1] > df.shape[0]:
    df = df.T

# Store results
results = []

# Loop through each cell type
for col in df.columns:
    data = df[col].sort_values().reset_index(drop=True)

    # Skip columns with too few valid values
    if data.count() < 4:
        continue
    mid = len(data) // 2
    low_expr = data.iloc[:mid]
    high_expr = data.iloc[mid:]

    # Means and stds
    means = [low_expr.mean(), high_expr.mean()]
    stds = [low_expr.std(), high_expr.std()]

    # T-test
    t_stat, p_val = ttest_ind(low_expr, high_expr)

    # Save results
    results.append({
        'Cell_Type': col,
        'Low_Mean': means[0],
        'High_Mean': means[1],
        'Low_Std': stds[0],
        'High_Std': stds[1],
        't_stat': t_stat,
        'p_val': p_val
    })
```

```
    # Plot
    lower_errors = [min(stds[0], means[0]), min(stds[1], means[1])]
    upper_errors = stds
    labels = ['Low', 'High']
    colors = ['skyblue', 'salmon']

    plt.figure(figsize=(4, 6))
    plt.bar(labels, means, yerr=[lower_errors, upper_errors], capsize=10, color=colors)
    plt.ylabel("Mean Proportion")
    plt.title(f"{col}: High vs Low Expression (Balanced)")
    plt.ylim(0, max([m + s for m, s in zip(means, stds)]) * 1.2)
    plt.tight_layout()
    plt.show()
```

#Heatmap

```
import os
import pandas as pd
os.chdir("/Users/kushaldas/Documents/cibersort_directory")  # Change this to the full path if
needed
expr_df = pd.read_csv("TET2_curated.tsv", sep="\t", index_col=0)
deg_df = pd.read_csv("TET2_high_FC_only.csv")
deg_genes = deg_df["Geneid"].dropna().unique()
# Subset expression matrix using DEG gene list
heatmap_df = expr_df.loc[expr_df.index.intersection(deg_genes)]
# Optional: merge DEG stats into expression data (for annotation or export)
merged_df = deg_df.set_index("Geneid").join(heatmap_df, how="inner")
# Save the final heatmap input matrix if needed
merged_df.to_csv("merged_heatmap_data.csv")
merged_df.to_csv("merged_heatmap_data.csv", index=True)
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv("merged_heatmap_data.csv", index_col=0)
metadata_cols = ['baseMean', 'log2FoldChange', 'stat', 'pvalue', 'padj']
expr_df = df.drop(columns=[col for col in metadata_cols if col in df.columns], errors='ignore')

# Step 3: Keep only numeric (expression) columns
expr_df = expr_df.select_dtypes(include=[np.number])

# Step 4: Row-wise Min-Max Normalization to [-1, +1]
```

```
expr_norm = 2 * (expr_df.sub(expr_df.min(axis=1), axis=0)
                 .div(expr_df.max(axis=1) - expr_df.min(axis=1), axis=0)) - 1

# Step 5: Define sample groups and colors (first 35 high, next 35 low)
sample_labels = ['TET2_High'] * 35 + ['TET2_Low'] * 35
col_colors = pd.Series(sample_labels, index=expr_df.columns).map({
    'TET2_High': 'red',
    'TET2_Low': 'blue'
})

ordered_columns = expr_df.columns[:35].tolist() + expr_df.columns[35:].tolist()
ordered_df = expr_norm.loc[:, ordered_columns]
ordered_colors = col_colors.loc[ordered_columns]

# Step 7: Plot the heatmap
g = sns.clustermap(
    ordered_df,
    cmap="vlag",
    figsize=(14, 10),
    col_cluster=False,
    row_cluster=True,
    col_colors=ordered_colors,
    xticklabels=False,
    yticklabels=False,
    vmin=-1,
    vmax=1
)
g.ax_heatmap.set_xlabel("")
g.ax_heatmap.set_ylabel("")
plt.show()
```